

2. Mi idea de app para el semestre — Plataforma de comunidad Esports

Problema: El ecosistema de esports está fragmentado: los aficionados siguen noticias y resultados en Twitter/Reddit, buscan equipo en servidores de Discord dispersos, y no existe un solo lugar donde puedan personalizar sus intereses (juegos, equipos profesionales, ligas) y a la vez encontrar compañeros de su mismo nivel para jugar torneos amateur.

Para quién: Estudiantes y aficionados a los esports en general (no solo de un juego), que quieren seguir su escena favorita y también armar equipo para competir.

MVP (3 funciones imprescindibles):

1. Selección de intereses/juegos favoritos al registrarse — el usuario elige los títulos que sigue (Valorant, LoL, CS2, Dota 2, FIFA, etc.) y equipos profesionales favoritos, para personalizar su feed de contenido (noticias, resultados, calendario de torneos pro).
2. Perfil de jugador con rango/rol por juego + matchmaking de compañeros — cada persona arma su perfil competitivo (juego principal, rango, rol) y puede buscar/filtrar otros jugadores compatibles para formar equipo, resolviendo el problema de la idea anterior dentro de esta plataforma más amplia.
3. Creación de equipo con calendario de scrims/partidas y recordatorios — una vez armado el equipo, pueden coordinar horarios de práctica o torneos, con notificación automática antes de cada partida.

3. Justificación de la tecnología

Mantengo desarrollo híbrido (Flutter o React Native), y en esta versión ampliada el argumento se refuerza:

- Personalización basada en datos, no en gráficos pesados: el feed de intereses, perfiles y matchmaking son básicamente listas y filtros sobre una base de datos; no requiere renderizado 3D ni acceso a hardware de bajo nivel, así que ir 100% nativo no se justifica en costo/beneficio.
- Consumo de múltiples APIs externas: al cubrir varios juegos, la app necesita integrarse con distintas APIs (Riot API para Valorant/LoL, Steam API para CS2/Dota 2, etc.); un stack híbrido con buen soporte HTTP/JSON facilita centralizar esa información sin duplicar lógica por plataforma.

- Notificaciones push como pieza central: tanto para noticias de los juegos seguidos como para recordatorios de scrims, se necesita push confiable — bien soportado por Firebase Cloud Messaging en Flutter/React Native.
- Feed de contenido dinámico: el módulo de "intereses" implica actualizar contenido con frecuencia (resultados, noticias); esto se maneja bien con paginación y caché en frameworks híbridos, sin sacrificar fluidez.
- Un solo código base, mayor alcance: la comunidad esports usa tanto Android como iOS indistintamente; cubrir ambas plataformas con un equipo de desarrollo pequeño (típico de un proyecto de semestre) es clave, y lo híbrido lo permite sin duplicar esfuerzo.
- Escalabilidad futura: si el proyecto creciera después del semestre (más juegos, chat de voz, streaming integrado), Flutter/React Native permiten ir incorporando módulos nativos puntuales sin reescribir toda la app desde cero.